ShakeBox

GS805 COFFEE MACHINE

Interface Guide

Android SDK (Kotlin)

Version 0.1

Date 2026-04-01

Author Anyeats World Co., Ltd.

Contact rnd@anyeats.co.kr

Classification Confidential

문서 이력

버전	날짜	내용
0.1	2026-04-01	초판 작성 — 전체 API (기본 14개 + 능동보고 4개 + R시리즈 2개 + Series 3/R 확장 11개)

목차

1. 개요

1.1 통신 사양

1.2 지원 모델

1.3 아키텍처

2. 설치 및 설정

3. 빠른 시작

4. API 레퍼런스

4.1 연결 관리

4.2 음료 제조 및 제어

4.3 정보 조회

4.4 유지보수

4.5 Series 3/R 확장 명령

4.6 커스텀 레시피 (R시리즈)

4.7 Event Flows

5. 사용 예제

5.1 레시피 정의 후 실행

5.2 채널 즉시 실행

6. 기술 사양

1. 개요

본 문서는 ShakeBox(GS805) 커피머신과 Android 디바이스 간 시리얼 통신을 위한 **GS805Serial Android SDK**의 인터페이스 가이드입니다. SDK는 Kotlin Coroutines + Flow 기반으로 설계되어 비동기 통신과 실시간 이벤트 스트리밍을 지원합니다.

1.1 통신 사양

항목	사양
통신 방식	UART RS232 (USB Serial)
전송 속도	9600 bps
데이터 형식	8N1 (8 data bits, No parity, 1 stop bit)
바이트 순서	Big Endian
응답 타임아웃	100ms (재시도 메커니즘 내장)

1.2 지원 모델

분류	시리즈	제어 시스템	해당 모델
Series 2	2nd	현마 (단일)	GS305, GS505, JK81, JK90, JK91
Series 3	3rd	**단일 스레드**	**GS505, GS805**, JK81, JK82, JK86
R Series	3rd	멀티 스레드	GS801, RD-JK86, JK88

> **참고:** ShakeBox(GS805)는 Series 3 (단일 스레드)입니다. R시리즈 전용 명령(레시피, 채널 실행)은 R시리즈 모델에서만 사용 가능합니다.

1.3 아키텍처

```
GS805Serial (Public API)
├── SerialManager
│   ├── SerialConnection (interface)
│   │   └── UsbSerialConnection (USB 구현)
│   ├── MessageParser (바이트 → ResponseMessage)
│   └── ReconnectManager (자동 재연결)
├── CommandQueue (순차 명령 큐)
```

```
├── GS805Logger (로깅)
└── MdbCashless (MDB 카드리더 - 별도 포트)
```

2. 설치 및 설정

Gradle 의존성 추가 (JitPack)

```
// settings.gradle.kts
dependencyResolutionManagement {
    repositories {
        maven("https://jitpack.io")
    }
}

// build.gradle.kts
dependencies {
    -- SDK 검증진행중, 검증 후 제공 --
    // implementation("com.github.anyeats:anyeats-***-***-***:1.0.0")
}
```

기술 스택

항목	사양
언어	Kotlin
비동기	Coroutines + Flow
USB 시리얼	usb-serial-for-android v3.7.3
최소 SDK	API 21 (Android 5.0)
빌드	Gradle KTS, AGP 8.2

3. 빠른 시작

```
// 1. 인스턴스 생성
val gs805 = GS805Serial(context)

// 2. 디바이스 검색 및 연결
val devices = gs805.listDevices()
gs805.connect(devices.first())

// 3. 음료 제조
gs805.makeDrink(DrinkNumber.HOT_DRINK_1)

// 4. 상태 조회
val status = gs805.getMachineStatus()
val balance = gs805.getBalance()
val errorInfo = gs805.getErrorInfo()

// 5. 이벤트 관찰 (Flow)
gs805.eventFlow.collect { event ->
    when (event.type) {
        MachineEventType.DRINK_COMPLETE -> println("음료 완성!")
        MachineEventType.CUP_DROP_SUCCESS -> println("컵 낙하 성공")
        else -> {}
    }
}

// 6. 연결 해제
gs805.disconnect()
gs805.dispose()
```

4. API 레퍼런스

4.1 연결 관리

메서드	설명
`listDevices()`	USB 시리얼 디바이스 목록 반환
`connect(device, config?)`	지정 디바이스에 연결
`connectToFirstDevice(config?)`	검색된 첫 번째 디바이스에 자동 연결
`connectByVidPid(vid, pid, config?)`	VID/PID를 지정하여 연결
`disconnect()`	현재 연결 해제

4.2 음료 제조 및 제어

메서드	설명
`makeDrink(drink, useLocalBalance?, timeout?)`	음료 제조 명령 전송
`setHotTemperature(upper, lower)`	온수 온도 설정 (60~99 °C)
`setColdTemperature(upper, lower)`	냉수 온도 설정 (2~40 °C)
`setCupDropMode(mode)`	컵 낙하 모드 설정
`setDrinkPrice(drink, price)`	음료 가격 설정 (0~99)

4.3 정보 조회

메서드	설명
`getMachineStatus()`	머신 상태 조회 (ready, busy, soldOut 등)
`getErrorCode()`	에러 코드 비트 플래그 조회
`getErrorInfo()`	에러 상세 정보 + 복구 가이드
`getBalance()`	투입 잔액 조회
`getSalesCount(drink)`	음료별 판매 수량 조회

4.4 유지보수

메서드	설명
`testCupDrop()`	컵 낙하 테스트
`autoInspection()`	자동 점검 실행
`cleanAllPipes()`	전체 파이프 세척
`cleanSpecificPipe(number)`	특정 파이프 세척 (번호 지정)
`returnChange()`	거스름돈 반환

4.5 Series 3/R 확장 명령

> 아래 명령은 Series 3 및 R시리즈 모델에서 사용 가능합니다. Series 2 모델에서는 지원되지 않습니다.

CMD	메서드	설명
단위 기능 테스트		
0x1A	`unitFunctionTest(testCmd, data1, data2, data3)`	분말/온수/좌표/컵/뚜껑/문/얼음 등 개별 기능 테스트
전자 잠금 장치		
0x1B	`lockDoor(lockNumber?)`	잠금 장치 잠금
	`unlockDoor(lockNumber?)`	잠금 장치 해제
	`getLockStatus(lockNumber?)`	잠금 상태 조회
물 보충		
0x1C	`waterRefill()`	물 보충 작업 실행
상태 조회 (상세)		
0x1E	`getControllerStatus()`	메인 컨트롤러 32비트 상세 상태 조회
0x1F	`getDrinkStatus()`	음료 제작 진행률 및 실패 코드 (EM01~EM26)
에러 조회 (객체별)		
0x22	`getObjectException(objectType)`	12가지 객체별 상세 에러 정보 조회
강제 종료		
0x23	`forceStopDrinkProcess()`	음료 제조 프로세스 강제 종료
	`forceStopCupDelivery()`	컵 전달 프로세스 강제 종료
컵 전달		
0x24	`cupDelivery(waitTimeSeconds)`	컵 전달 (문 개방 후 지정 시간 대기)

4.6 커스텀 레시피 (R시리즈 전용)

> **주의:** 아래 명령은 R시리즈(GS801, RD-JK86, JK88) 전용입니다. GS805(Series 3)에서는 지원되지 않습니다.

CMD	메서드	설명
0x1D	`setDrinkRecipeProcess(drink, steps)`	커스텀 음료 레시피 정의 (최대 8단계)
0x25	`executeChannel(channel, waterType, ...)`	단일 채널 즉시 실행

4.7 Event Flows

SDK는 Kotlin Flow 기반의 실시간 이벤트 스트리밍을 제공합니다.

Flow	타입	설명
`messageFlow`	`SharedFlow<ResponseMessage>`	모든 응답 메시지 (raw)
`eventFlow`	`Flow<MachineEvent>`	머신 이벤트 (능동 보고)
`connectionStateFlow`	`StateFlow<Boolean>`	연결 상태 변화
`reconnectEventFlow`	`SharedFlow<ReconnectEvent>`	자동 재연결 이벤트

능동 보고 이벤트 (MachineEventType)

CMD	이벤트	설명
0x05	`CUP_DROP_SUCCESS`	컵 낙하 성공
0x10	`DRINK_COMPLETE`	음료 제조 완료
0x06	`ICE_DROP_COMPLETE`	얼음 투하 완료
0x20	`TRACK_OBSTACLE`	트랙 장애물 감지

5. 사용 예제

5.1 레시피 정의 후 실행 (R시리즈)

```
// 레시피 정의: 컵 배출 → Powder1 + 물 200mL → Powder2
val steps = listOf(
    RecipeStep.cupDispense(dispenser = 1),
    RecipeStep.instantChannel(
        channel = 0, waterType = WaterType.HOT,
        materialDuration = 1000, waterAmount = 2000,
        materialSpeed = 50, mixSpeed = 30,
    ),
    RecipeStep.instantChannel(
        channel = 1, waterType = WaterType.HOT,
        materialDuration = 500, waterAmount = 0,
        materialSpeed = 50, mixSpeed = 30,
    ),
)
gs805.setDrinkRecipeProcess(DrinkNumber.HOT_DRINK_1, steps)
gs805.makeDrink(DrinkNumber.HOT_DRINK_1)
```

5.2 채널 즉시 실행 (R시리즈)

```
// 컵 낙하 후 채널 순차 실행
gs805.testCupDrop()
gs805.executeChannel(
    channel = 0, waterType = WaterType.HOT,
    materialDuration = 1000, waterAmount = 2000,
    materialSpeed = 50
)
gs805.executeChannel(
    channel = 1, waterType = WaterType.HOT,
    materialDuration = 500, waterAmount = 0,
    materialSpeed = 50
)
```

6. 기술 사양

항목	사양
언어	Kotlin 1.9+
비동기 처리	Kotlin Coroutines + Flow
USB 시리얼 라이브러리	usb-serial-for-android v3.7.3
최소 Android SDK	API 21 (Android 5.0 Lollipop)
빌드 시스템	Gradle KTS, Android Gradle Plugin 8.2
통신 프로토콜	UART RS232, 9600 bps, 8N1, Big Endian
명령 큐	순차 처리 (CommandQueue), 자동 재시도
자동 재연결	Exponential Backoff / Fixed Interval
패키지	`kr.co.anyeats.gs805serial`

© 2026 Anyeats World Co., Ltd. All rights reserved. — Confidential